

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1-EXPERIMENTS
Weightage:	45%	Total Marks:	25
CO3 Statement:	Register transfers, Arithmetic and logic operations. Multiple Bus Organization: Data transfer among multiple buses. Pipelining: Stages of pipelining, Pipeline performance, hazards (data, instruction). (BL3)		
Student Name:	PAKSHAYA	Reg.No.:	192525190
Department:	AIML	Date:	18/8/26

ANNEXURE A

Experiment Question

- 1 Design and implement a Register Transfer Unit (RTU) to demonstrate data transfer between registers. Perform operations such as $R2 \leftarrow R1$, $R3 \leftarrow R1 + R2$, and $R4 \leftarrow R3$. Verify the contents of registers after each transfer.
- 2 Design and implement an Arithmetic and Logic Unit (ALU) to perform arithmetic operations such as addition and subtraction and logical operations such as AND, OR, XOR, and NOT. Display and verify the output for different input combinations.
- 3 Implement a Multiple Bus Organization for a CPU data path. Demonstrate the transfer of data among registers through multiple buses and perform an arithmetic operation using the ALU. Verify the register contents after each operation.
- 4 Design and simulate a 4-stage instruction pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), and Write Back (WB). Demonstrate the execution of multiple instructions simultaneously and calculate the pipeline speedup.
- 5 Implement and analyze pipeline hazards using a set of dependent instructions. Demonstrate data hazards and instruction hazards, and apply appropriate techniques such as stalling, forwarding, or branch handling to resolve them. Compare the pipeline performance before and after hazard resolution.

Note: use Logisim

1. 1. Register Transfer Unit (RTU)

Aim

To design and implement a Register Transfer Unit to demonstrate data transfer between registers and verify the register contents after each operation.

Operations

- $R2 \leftarrow R1$
- $R3 \leftarrow R1 + R2$
- $R4 \leftarrow R3$

Verilog Code

```
module RTU;
```

```
reg [7:0] R1, R2, R3, R4;
```

```
initial begin
```

```
    // Initial values
```

```
    R1 = 8'd10;
```

```
    R2 = 8'd0;
```

```
    R3 = 8'd0;
```

```
    R4 = 8'd0;
```

```
    $display("Initial: R1=%d R2=%d R3=%d R4=%d", R1,R2,R3,R4);
```

```
    // R2 <- R1
```

```
    R2 = R1;
```

```
    $display("After R2 <- R1: R1=%d R2=%d R3=%d R4=%d",  
            R1,R2,R3,R4);
```

```
    // R3 <- R1 + R2
```

```
    R3 = R1 + R2;
```

```
    $display("After R3 <- R1 + R2: R1=%d R2=%d R3=%d R4=%d",  
            R1,R2,R3,R4);
```

```
    // R4 <- R3
```

```
    R4 = R3;
```

```
    $display("After R4 <- R3: R1=%d R2=%d R3=%d R4=%d",  
            R1,R2,R3,R4);
```

```
    $finish;
```

```
end
```

```
endmodule
```

Expected Output

Operation	R1	R2	R3	R4
Initial	10	0	0	0
$R2 \leftarrow R1$	10	10	0	0

Operation	R1	R2	R3	R4
$R3 \leftarrow R1 + R2$	10	10	20	0
$R4 \leftarrow R3$	10	10	20	20

Result

The Register Transfer Unit successfully performs the required register transfers and arithmetic operation. The final contents are:

$R1 = 10$, $R2 = 10$, $R3 = 20$, $R4 = 20$.



2. Arithmetic and Logic Unit (ALU)

Aim

To design and implement an ALU that performs arithmetic operations such as addition and subtraction and logical operations such as AND, OR, XOR, and NOT.

Operations

Opcode	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	NOT A

Verilog Code

```
module ALU;
```

```
reg [7:0] A, B;
```

```
reg [2:0] OP;
```

```
reg [7:0] Y;
```

```
always @(*) begin
```

```
case(OP)
```

```
3'b000: Y = A + B;
```

```
3'b001: Y = A - B;
```

```
3'b010: Y = A & B;
```

```
3'b011: Y = A | B;
```

```

        3'b100: Y = A ^ B;
        3'b101: Y = ~A;
        default: Y = 8'b00000000;
    endcase
end

```

```

initial begin
    A = 8'b00001111; // 15
    B = 8'b00000011; // 3

    OP = 3'b000; #10;
    $display("ADD : %d", Y);

    OP = 3'b001; #10;
    $display("SUB : %d", Y);

    OP = 3'b010; #10;
    $display("AND : %d", Y);

    OP = 3'b011; #10;
    $display("OR : %d", Y);

    OP = 3'b100; #10;
    $display("XOR : %d", Y);

    OP = 3'b101; #10;
    $display("NOT : %d", Y);

    $finish;
end

```

endmodule

Expected Output

For:

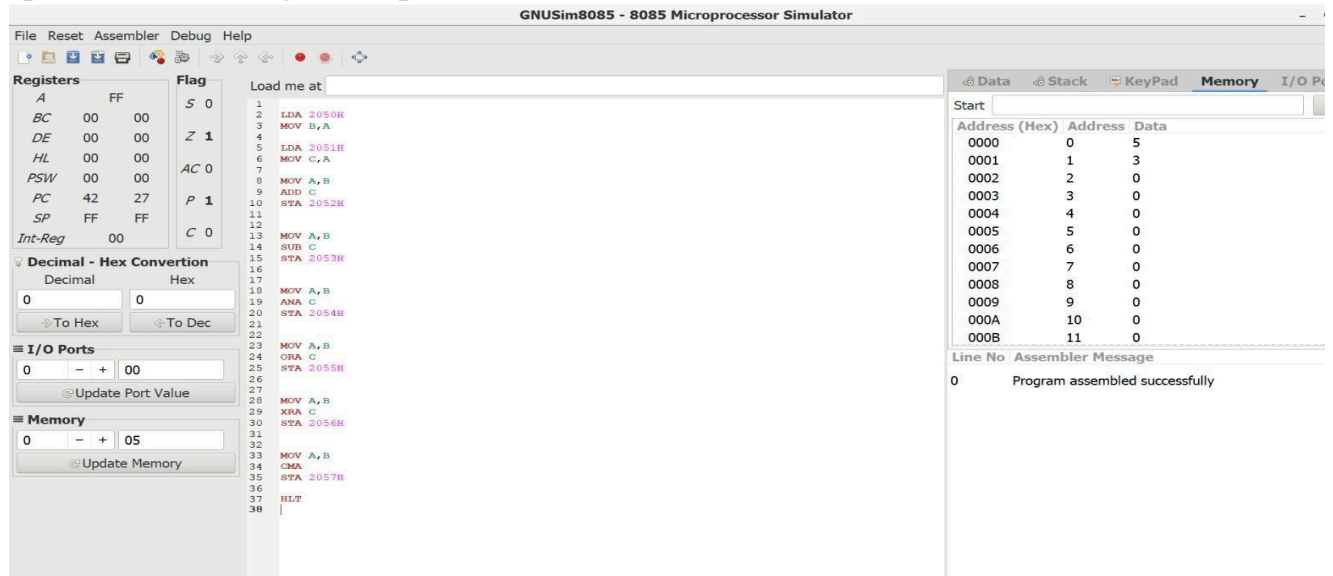
- A = 15 = 00001111
 - B = 3 = 00000011
- | Operation | Result |
|-----------|--------|
| A + B | 18 |
| A - B | 12 |
| A AND B | 3 |
| A OR B | 15 |
| A XOR B | 12 |
| NOT A | 240 |

NOT A = 11110000 = 240 for an 8-bit ALU.

Result

The ALU successfully performs addition, subtraction, AND, OR, XOR, and NOT

operations for the given input combinations.



3. Multiple Bus Organization

Aim

To implement a multiple-bus organization for a CPU datapath, demonstrate data transfer between registers through multiple buses, and perform an arithmetic operation using the ALU.

Basic Organization

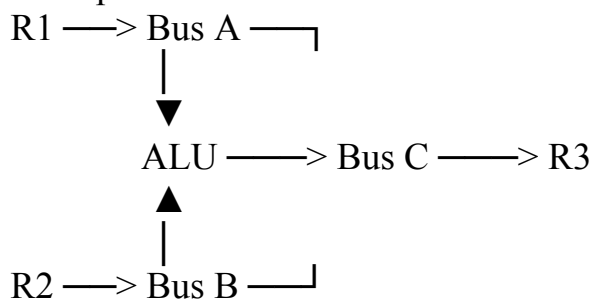
A simple three-bus datapath consists of:

- Bus A → carries first ALU operand
- Bus B → carries second ALU operand
- Bus C → carries ALU result back to a register

Example operation:

$R3 \leftarrow R1 + R2$

The operation occurs as:



Verilog Code

```
module Multiple_Bus;
```

```
reg [7:0] R1, R2, R3, R4;
```

```
reg [7:0] BusA, BusB, BusC;
```

```
initial begin
```

```
// Load registers
```

```
R1 = 8'd15;
```

```
R2 = 8'd5;
```

```

R3 = 8'd0;
R4 = 8'd0;

$display("Initial: R1=%d R2=%d R3=%d R4=%d",
        R1,R2,R3,R4);

// R1 -> Bus A
BusA = R1;

// R2 -> Bus B
BusB = R2;

// ALU addition
BusC = BusA + BusB;

// Bus C -> R3
R3 = BusC;

$display("After R3 <- R1 + R2:");
$display("R1=%d R2=%d R3=%d R4=%d",
        R1,R2,R3,R4);

// R3 -> R4
BusA = R3;
R4 = BusA;

$display("After R4 <- R3:");
$display("R1=%d R2=%d R3=%d R4=%d",
        R1,R2,R3,R4);

$finish;
end

endmodule

```

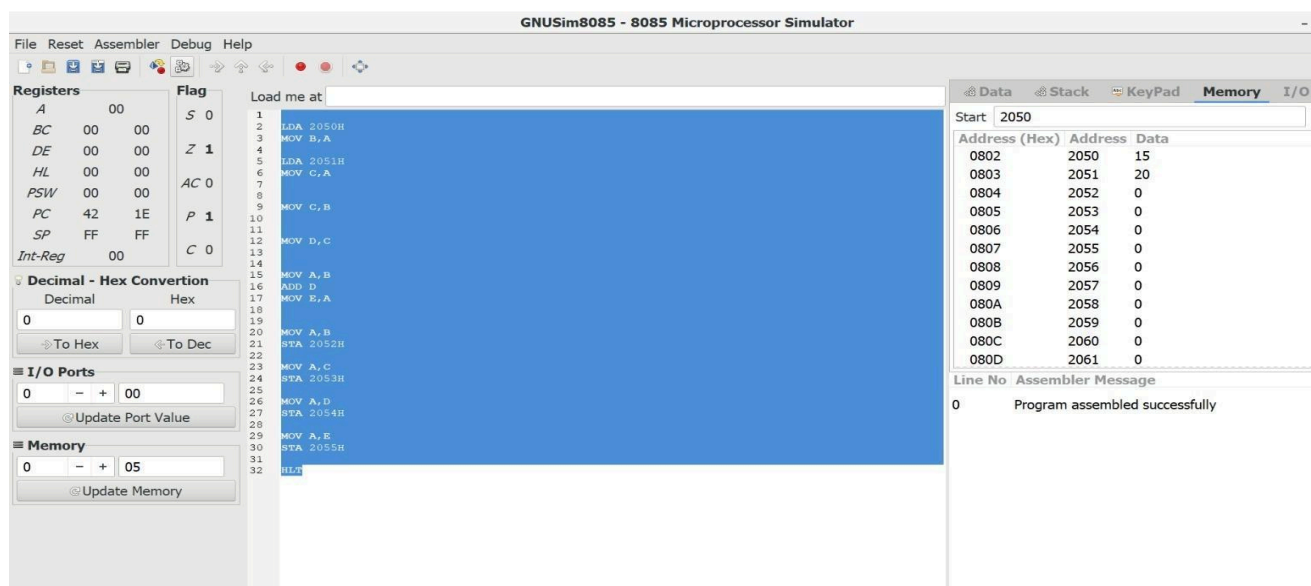
Expected Output

Operation	R1	R2	R3	R4
Initial	15	5	0	0
$R3 \leftarrow R1 + R2$	15	5	20	0
$R4 \leftarrow R3$	15	5	20	20

Result

The multiple-bus CPU datapath successfully transfers data through Bus A, Bus B, and Bus C. The ALU performs the addition and stores the result in R3.

Final values: R1 = 15, R2 = 5, R3 = 20, R4 = 20.



4. Four-Stage Instruction Pipeline

Aim

To design and simulate a four-stage instruction pipeline consisting of:

1. IF – Instruction Fetch
2. ID – Instruction Decode
3. EX – Execute
4. WB – Write Back

and calculate the pipeline speedup.

Pipeline Stages

IF → ID → EX → WB

Consider four instructions:

I1

I2

I3

I4

Pipeline Timing

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	WB	EX	ID	IF
5		WB	EX	ID
6			WB	EX
7				WB

Therefore, 4 instructions require 7 cycles using the pipeline.

Non-Pipelined Execution

Without pipelining, each instruction requires 4 stages:

4 instructions × 4 stages = 16 cycles

Pipelined Execution

For n instructions and k stages:

$$T_{\text{pipeline}} = k + (n - 1)$$

For $k = 4$ and $n = 4$:

$$T_{\text{pipeline}} = 4 + (4 - 1) = 7 \text{ cycles}$$

Speedup

$$\text{Speedup} = \frac{T_{\text{non-pipeline}}}{T_{\text{pipeline}}}$$

$$\text{Speedup} = \frac{16}{7}$$

$$\text{Speedup} = 2.286$$

So the pipeline gives approximately $2.29\times$ speedup for four instructions.

Simple Verilog Simulation

```
module Pipeline;
```

```
integer cycle;
```

```
initial begin
```

```
    $display("Cycle\tIF\tID\tEX\tWB");
```

```
    for(cycle = 1; cycle <= 7; cycle = cycle + 1) begin
```

```
        case(cycle)
```

```
            1: $display("1\tI1\t-\t-\t-");
```

```
            2: $display("2\tI2\tI1\t-\t-");
```

```
            3: $display("3\tI3\tI2\tI1\t-");
```

```
            4: $display("4\tI4\tI3\tI2\tI1");
```

```
            5: $display("5\t-\tI4\tI3\tI2");
```

```
            6: $display("6\t-\t-\tI4\tI3");
```

```
            7: $display("7\t-\t-\t-\tI4");
```

```
        endcase
```

```
    end
```

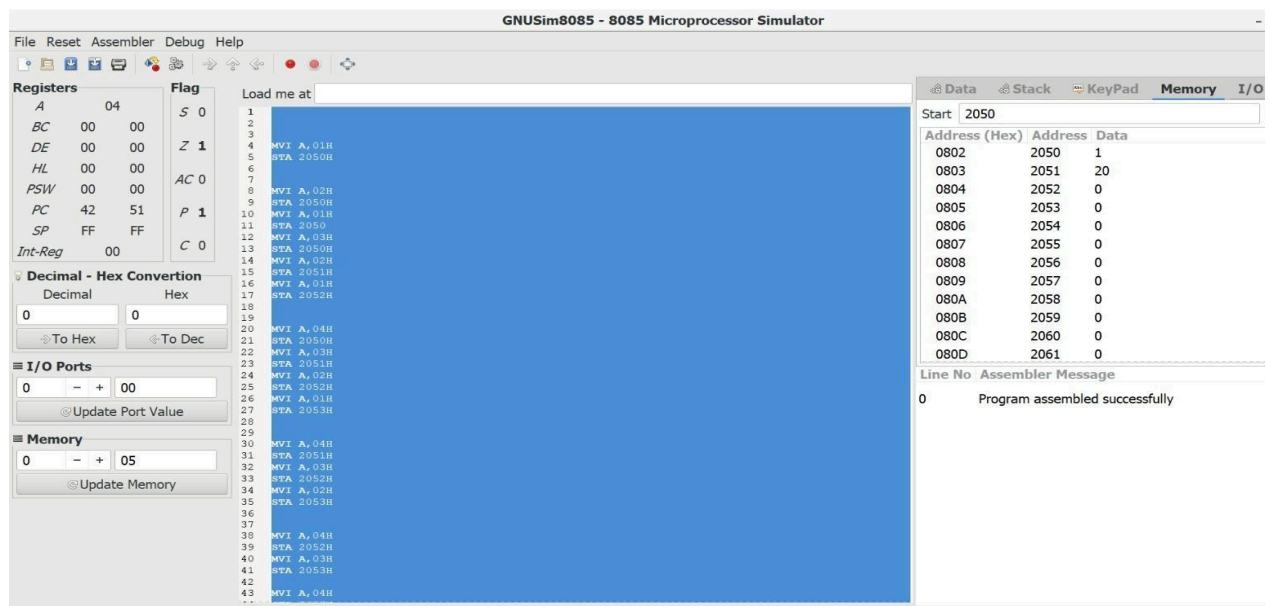
```
    $finish;
```

```
end
```

```
endmodule
```

Result

The four-stage pipeline successfully executes multiple instructions simultaneously. For four instructions, execution time is reduced from 16 cycles to 7 cycles, giving a speedup of approximately $2.29\times$.



5. Pipeline Hazards and Hazard Resolution

Aim

To implement and analyze pipeline hazards using dependent instructions and demonstrate techniques such as stalling, forwarding, and branch handling.

Types of Hazards

1. Data Hazard

Consider:

I1: $R1 \leftarrow R2 + R3$

I2: $R4 \leftarrow R1 + R5$

I2 requires the value of R1 produced by I1.

This creates a RAW (Read After Write) data hazard.

Without Forwarding

The pipeline may have to stall:

Cycle	I1	I2
1	IF	
2	ID	IF
3	EX	ID
4	MEM/EX	STALL
5	WB	STALL
6		EX
7		WB

The stall increases execution time.

2. Forwarding Technique

Instead of waiting for the value to be written back to the register, the ALU result from I1 can be directly forwarded to the ALU input of I2.

I1 ALU Result



Forwarding Unit





I2 ALU Input

This reduces or eliminates unnecessary stalls.

Example

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

The result of I1 is forwarded directly to I2.

3. Instruction/Control Hazard

A branch instruction creates a control hazard because the processor does not immediately know which instruction should be fetched next.

Example:

I1: BEQ R1, R2, TARGET

I2: ADD R3, R4, R5

I3: SUB R6, R7, R8

If the branch is taken, instructions already fetched after the branch may be incorrect.

Branch Handling

A common solution is to:

5. Stall until the branch decision is known, or
6. Flush incorrectly fetched instructions, or
7. Use branch prediction.

For a simple pipeline, stalling and flushing can be used.

Pipeline Performance Comparison

Assume 5 instructions require 9 cycles without hazards.

If two stall cycles are introduced:

$$T_{hazard} = 9 + 2 = 11 \text{ cycles}$$

With forwarding, the two unnecessary stalls can be removed:

$$T_{forwarding} = 9 \text{ cycles}$$

Comparison

Method	Cycles	Performance
Normal pipeline	9	Best
With stalls	11	Lower
With forwarding	9	Improved
With branch flush	Depends on branch	Reduced

Simple Verilog Example

```
module Pipeline_Hazard;
```

```
reg [7:0] R1, R2, R3, R4, R5;
```

```
initial begin
```

```
    R2 = 8'd10;
```

```
    R3 = 8'd20;
```

```
    R5 = 8'd5;
```

```

// I1: R1 = R2 + R3
R1 = R2 + R3;

// I2: R4 = R1 + R5
// Forwarded value of R1 is used
R4 = R1 + R5;

$display("After I1: R1 = %d", R1);
$display("After I2: R4 = %d", R4);

$finish;
end

```

endmodule

Expected Output

After I1: R1 = 30

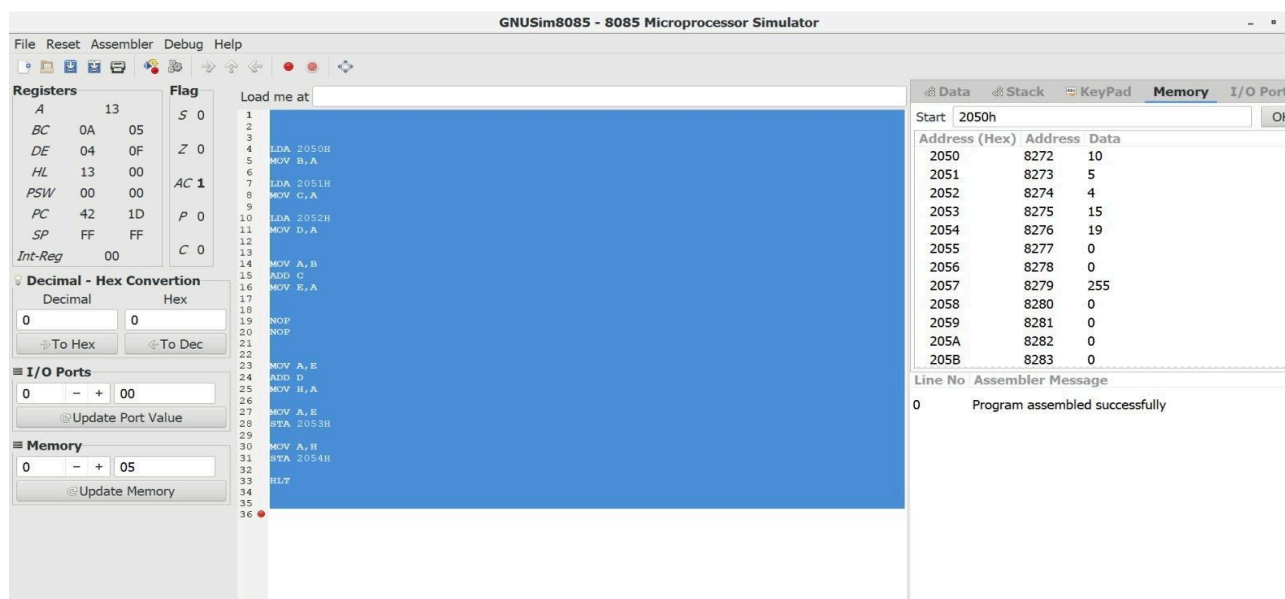
After I2: R4 = 35

Result

The pipeline hazards were successfully identified and resolved. Data hazards were handled using forwarding, while instruction/control hazards can be handled using stalling, flushing, or branch prediction. Forwarding improves performance by reducing unnecessary pipeline stalls.

Final Viva/Record Summary

Experiment	Main Concept	Final Result
1. RTU	Register transfer	R2=R1, R3=R1+R2, R4=R3 verified
2. ALU	Arithmetic & logic	ADD, SUB, AND, OR, XOR, NOT verified
3. Multiple Bus	CPU datapath	Data transferred through multiple buses and ALU operation verified
4. Pipeline	IF-ID-EX-WB	4 instructions completed in 7 cycles; speedup = 2.29×
5. Hazards	Stalling & forwarding	Data/control hazards identified and resolved



Code of Conduct:

I P.AKSHAYA 192525190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation